Proving a program leaks nothing

A step-by-step walkthrough of the SPS confidentiality checker, its evidence pipeline, and its LLVM/MLIR test harness

Claim-ledger lecture notes

Abstract

This is a walkthrough of what it takes to answer one question mechanically: *does this compiled artifact reveal a secret to an observer who is not entitled to it?* The answer is never a single proof. It is a chain of reductions, each trading one obligation for a different obligation that is easier to attack, and the interesting content of the system lives in the trades rather than in any individual step.

The right margin of every step carries a *claim ledger*: the obligations still open at that point in the argument, colour-coded by what the step just did to them. A reader who follows only the margin still learns the shape of the argument. The last step leaves the ledger empty.

Part 4 then maps every step onto the regression fixtures that guard it, and Part 5 answers the question that motivates all of it: how would you know the checker itself is correct, and what would a mechanized proof for an MLIR-based analysis actually look like?

Part 6 then confronts the finding that undoes much of the rest: the artifact a checker reads is not the artifact that runs. Twenty reproduced compiler transformations force an architecture in which policy stops travelling downward and observations travel upward instead.

Part 7 is the shortest and the only one about something that runs. Six claims in the parts above were wrong until a tool was executed against them, which is the argument for building the smallest real thing before designing any further.

Contents

1 Orientation

1.1 The statement

Fix a compiled artifact P. Partition its inputs into public inputs p and secret inputs s. Fix an observer and let Obs_Θ be the projection of an execution trace onto what that observer can actually see under target profile Θ — branch directions, memory addresses, operation latencies and counts, allocation sizes, transfer lengths, values arriving at public sinks, and the identity of the host or audience a message is delivered to.

Some dependence on the secret is intended: a service that returns nothing derived from its inputs is useless. So the property is not plain noninterference but *release-relative* noninterference. Let R be the releases authorized for that observer. Then:

$$\forall p,\ s_0,\ s_1 : \quad R(p, s_0) = R(p, s_1) \implies \mathrm{Obs}_\Theta\big(\mathrm{Tr}(P, p, s_0)\big) = \mathrm{Obs}_\Theta\big(\mathrm{Tr}(P, p, s_1)\big). \tag{1}$$

Read it as a challenge from an adversary. They choose the public inputs once and two secrets freely. They only have to respect one constraint: whatever they are *entitled* to learn must come out the same both times. If they can then make any observable differ, they have learned something they were not entitled to, and P leaks.

Remark 1.1. Equation (1) quantifies over *pairs* of executions. That single fact drives almost every design decision in the rest of this document, and Part 2 is devoted to its consequences.

Two warnings about the form above, both discharged there. It is a statement of *intent*, not an encoding: installing its antecedent as an initial constraint on the pair is a known unsoundness, and Part 2.3 gives the prefix-causal construction that replaces it. And P here is informal shorthand; from Step 1 onward the object under analysis is the frozen artifact T together with its pipeline I, because a verdict is only ever a statement about specific bytes.

1.2 Why functional correctness is not enough

A verified compiler such as CompCert proves that the target program computes the same *function* as the source. That is a statement about one execution at a time, and confidentiality is not a property of one execution at a time.

Concretely, every one of the following transformations preserves the computed function and destroys (1):

- replacing a constant-time conditional move with a branch, so the observer learns the condition from control flow;
- strength-reducing a division into a multiply-high, changing which operations are executed and how long they take;
- eliminating a store that was scrubbing a secret, because nothing reads it afterwards;
- promoting a secret buffer into registers that are later spilled;
- choosing an allocation size that depends on a secret count.

So a confidentiality checker cannot be a corollary of a correctness checker. It needs its own property, its own observation model, and — because the property is about pairs — its own machinery.

1.3 Four outcomes, and why three would be dishonest

The checker reports one of four values for each question it is asked:

Outcome	Meaning	Obligations
VERIFIED	no residual forbidden dependence at the claimed boundary	must be none
UNSAFE	a forbidden dependence, witnessed by a replayable counterexample	must be none
UNKNOWN	required semantics or a contract is absent	must name what is missing
CONDITIONAL	acceptable only if named external assumptions hold	must name every assumption

The two middle-ground values carry the weight. A checker that can only say *safe* or *unsafe* must guess when it meets a function pointer, a recursive call, an opaque external callee, an uninitialized read, or a solver timeout — and a guess in the VERIFIED direction is exactly the failure that makes such a tool worthless. UNKNOWN is what lets the analysis fail closed. CONDITIONAL is what lets it be honest about facts that live outside the model altogether: real helper latency, backend trace preservation, hardware isolation.

Remark 1.2 (The invariant that makes the ledger work)**.** VERIFIED and UNSAFE must carry *no* outstanding obligations, while UNKNOWN and CONDITIONAL must name theirs. That is precisely the discipline a claim ledger encodes: an outcome is only allowed to be terminal when the ledger for it is empty.

1.4 Evidence levels

The system separates *where* a fact is established from *how much* it is worth. Five levels are used throughout, and every fixture in the harness names the ones it relies on.

Level	What it supplies
L0	declarations: secrets, observer, release policies, helper contracts, hosts, target facts
L1	propagation: dependence and effects — branches, addresses, latencies, sinks
L2	a relational witness: two lanes coupled at entry, different secrets, different observations
L3	attribution: relating source, imported IR, and target artifact to a compiler boundary
L4	facts outside the modeled semantics: real latency, compressors, hardware isolation, certificates

L0 is a *contract, not a proof of the contract.* If no latency summary for a helper is available, the honest result is UNKNOWN; selecting a named operand-dependent test profile lets L1 classify the call as UNSAFE under that profile, and L4 is still required before claiming a deployed helper matches it.

1.5 How to read the ledger

Each step in Part 3 carries a panel in the right margin listing the obligations *still open* after that step. Rows are coloured by what the step did:

NEW (green)	the step opened a new obligation
REDUCED (amber)	the step rewrote an obligation into a different one
USED (amber)	the step relied on it; the body is unchanged
DISCHARGED (red)	the step closed it; struck out once, then dropped
carried (grey)	untouched by this step, still open

Two reading conventions. First, an obligation quantified over an index — per coalition, per site, per entry — is *one* row carrying a subscript, not one row per instance; otherwise the panel would be unreadable by Step 3. Second, the bodies are deliberately terse because a struck-out body cannot break across lines. The abbreviations are:

$\mathrm{Sec}_A(P)$	P satisfies (1) for coalition A
$\mathrm{Bind}(T, I)$	every result is bound to this exact artifact and pipeline
$\mathrm{Total}(\mathcal{M})$	the policy manifest is total over reachable sites
$\mathrm{NFConf}(T, I)$	the artifact lies inside the analyzable fragment
$\mathrm{Adm} \neq \emptyset$	at least one execution satisfies the declared preconditions
$\mathrm{PairDom}_A \neq \emptyset$	at least one admitted low-equal *pair* exists
DiagOK	the diagnostic layer assigned a disposition to every site
$\mathrm{ProdSafe}_A$	the exact two-lane product cannot reach a bad state
$\mathrm{Replay}_A(w)$	witness w replays under the exact semantics
$\mathrm{PairRef}_A$	the backend preserves the observable trace

The arc opens with (1) itself and ends with an empty ledger. If you only read the margins, you will still see which obligation each step traded away and what it bought.

2 Background: pairs, products, and 2-safety

This part builds the one piece of machinery the rest of the document leans on. If you are fluent in information-flow policy but have not thought much about *how* a relational property gets checked mechanically, this is the gap worth closing before Part 3.

2.1 Safety properties and their limits

Most program verification concerns *safety* properties: a predicate over a single execution, of the form "no execution ever reaches a bad state." Null dereference,

array bounds, assertion violation, and functional correctness against a specification are all of this shape. The reason this class is comfortable is that it is checkable by reachability — build an abstraction of the state space, show the bad set is unreachable, done. Every mature tool we have, from abstract interpreters to model checkers to SMT-backed verifiers, is built for it.

Equation (1) is not of this shape. No single execution of P is a counterexample to it. A violation is a *pair*: two executions that agree on everything the observer is entitled to know, and disagree on some observable. This is the defining feature of a *2-safety* property, also called a hyperproperty of arity two.

Remark 2.1 (Read (1) as a specification, not as an encoding)**.** Equation (1) is stated as a whole-run implication, and it is correct as an informal statement of intent. It is **not** how the checker encodes the question, and turning it directly into a solver query is a known unsoundness. Part 2.3 explains why and gives the construction that replaces it.

Remark 2.2. The practical consequence is worth stating plainly: you cannot instrument P, run it, and check an assertion. There is no assertion over a single run whose failure means "leak." Dynamic taint tracking, which is what LLVM's DataFlowSanitizer does, therefore cannot decide (1); it approximates it by tracking labels through one execution and reporting when a labelled value reaches a sink. That approximation has no declassification and no notion of timing, which is why it is the wrong model here even though its label plumbing is instructive.

2.2 The product construction

There is a standard and very useful move: **2-safety of P is ordinary safety of a program that runs two copies of P at once.** Build $P \times P$, the *product* or *self-composition*, whose state is a pair of P states, one per *lane*. Couple the initial states on what the observer can see *at entry*, let the secrets range freely, and add a bad-state predicate that fires when the lanes' observations diverge.

Precisely which facts couple the initial states matters a great deal, and is the subject of Part 2.3. For now note only what they are *not*: they do not include equality of any release the program has not performed yet.

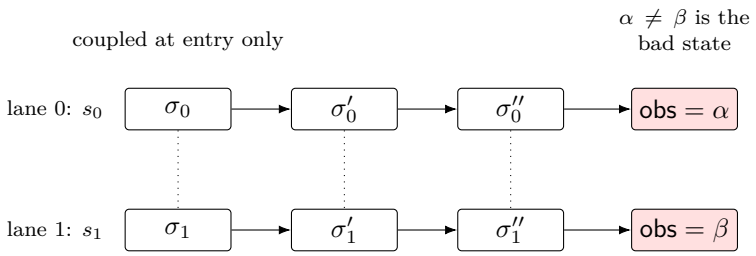

Figure 1: The two-lane product. Initial states are coupled on what is visible *at entry* — public configuration, public alias topology, and the initially-visible components — with the secrets free. Releases are *not* part of this coupling; they are handled step by step as the lanes run (Part 2.3). Dotted lines are the lockstep alignment. A violation is exactly a reachable state where the lanes' projected observations differ, which is an ordinary reachability question over the product.

This is the hinge of the design. It converts a property no existing tool can check directly into one that every existing tool can check, at the cost of squaring the state space. It is also why the specification speaks of an "audit-all whole-entry product" rather than a type system: the product is the *authoritative* object, and a verdict is a reachability result over it.

2.3 The release ledger, and why whole-run equality is wrong

Remark 2.3 (Two things called a ledger)**.** This document has a *claim ledger* in the margin — a presentational device of these notes. The specification separately has a *release ledger*, an object inside the product semantics. They are unrelated. This subsection is about the second; every margin panel is the first.

Remark 2.1 promised an explanation. Here it is, because this is the one place where the obvious encoding of (1) is not merely imprecise but *unsound*.

What couples the initial states. The lanes must agree on what the coalition can see *at entry*: the public configuration, the public alias topology, and the read-carrier of every initially-visible component. Mechanism and timing-environment choices are coupled too, so a pair cannot differ on a modelled implementation choice. Deliberately absent: any requirement that the secret components differ, and — the point of this subsection — any equality of a release the program has not performed yet.

Why that last omission matters. (1) reads: if the authorized releases come out equal, the observations must come out equal. The obvious encoding installs $R(p, s_0) = R(p, s_1)$ as an initial constraint on the product and then asks for bad-state reachability. A two-instruction program shows that this is wrong:

```
emit(public_channel, h)   ;  release(policy, audience, h)
```

The secret is published at step 1 and only afterwards passed through an authorized release at step 2. Under the whole-run encoding the constraint keeps only lane pairs whose complete release histories agree — and since the released value *is* h, that means only pairs with equal h. Those pairs trivially agree at step 1 too. The leak is never examined: the encoding has filtered away precisely the pairs that demonstrate it. Note the direction of the error — it is permissive, not conservative.

The fix is causal. Releases are consulted step by step as the lanes run, never installed up front:

At an aligned step	The ledger does
no release here	require the projected words to agree, as usual
release authorized for this observer, values equal	append the entry and continue; the obligation stays active
release authorized for this observer, values differ	permitted — but everything outside the release's declared footprint must still agree, and the pair's obligation then *retires*
release not authorized for this observer	conceal the value; require everything else about the event to agree

The transition takes only the current ledger and the current pair of events. It has no parameter through which a later event can be inspected, so a release that has not happened cannot excuse an observation that already has.

Remark 2.4. "Retires" is doing real work: an authorized difference ends the obligation for that pair *from that point*, rather than licensing differences retroactively. A pair is protected up to its first authorized divergence and released afterwards, which is what makes declassification expressible without making it a loophole.

This is not hypothetical. The program above is encoded in the harness as `prefix_causal_release.bad.mlir`, and a checker built on the whole-run encoding reports that fixture safe.

2.4 Why the product is harder than it looks

Figure 1 is drawn with the lanes in lockstep, which quietly assumes they follow the same control path. They need not. Once a secret can influence control flow the lanes desynchronize, and the product must say something about misaligned executions. This is where the bad-state predicate stops being "the outputs differ" and becomes a disjunction over structural divergences:

1. one lane can step and the other cannot;
2. the two lanes' next control locations differ;
3. their termination or failure statuses differ;
4. their event site or occurrence alignment differs;
5. the projected event words they emit differ.

Items 1–4 are *structural*: they compare the shape of the two executions, not their data. That distinction matters because structural divergence is not weakened when the observer's projection conceals a payload. An observer who cannot read a value can still count operations, notice that a branch was taken, and see that a run terminated early.

Remark 2.5 (Where the subtle bugs live)**.** Nearly every unsoundness in this area is a case where somebody checked item 5 and forgot items 1–4, or checked a *local* version of item 4 rather than a global one. Two of the countermodels in Part 3 are exactly of this form, and one of them — Step 7 — is a program where every per-site payload and count agrees and only the global ordering differs.

2.5 Two layers, and the discipline between them

Squaring the state space is expensive, so it is tempting to avoid the product with a cheap syntactic analysis: label each value Low or High, join labels at merges, and complain when High reaches a sink. That is a classic information-flow type system, and it is genuinely useful. But note carefully which way its guarantee points. For a *sound over-approximation* it is **acceptance** that is conclusive — the classical theorem is that a well-typed program *is* noninterferent — while its *rejections* may be false alarms. A type system proves; it cannot refute.

Two things nevertheless remove that proving power in this setting. The observation model here includes timing, addresses, allocation sizes, and operation counts, and a label discipline over values says nothing about any of them. And the property is release-relative, so a declassification must be *permitted* at exactly the

right point, which the prefix-causal ledger of Part 2.3 decides and a label lattice cannot express. What survives is a useful *triage*: cheap, sound in the direction of refusal, and unable to close the question.

The architecture therefore has two layers with a deliberately asymmetric contract:

Diagnostic layer (L1)	one Low/High pass, with no proof-authoritative strong update, no function summaries, and no slice selection. Deliberately weak.
Product layer (L2)	the exact two-lane product over the whole entry. Authoritative.

The contract is: **the diagnostic layer may narrow the work, and may never discharge an obligation.** Its weakness is the point. Because it cannot perform a strong update it cannot conclude that a secret store was later overwritten; because it has no summaries it cannot conclude anything about a callee. So it cannot be unsound — it can only be imprecise, and its correct output at a site it cannot decide is RELATIONAL REQUIRED: *the product must decide this one.*

Remark 2.6 (The single most dangerous repair)**.** When the diagnostic layer reports a violation on a program that is in fact safe, the tempting fix is to teach it the missing reasoning — a strong update, a summary, a slice. Every such addition moves proof burden into the layer designed to carry none, and the specification forbids the consequence outright: no diagnostic disposition, including *statically discharged* or *not observable*, may establish safety or remove a product constraint. This is why the harness ships *paired anti-controls* rather than controls alone, a discipline developed in Part 4.3.

3 The walkthrough

Ten steps take (1) from an informal wish to an empty ledger. Read the margin panels as the primary text if you like; the prose exists to justify each trade.

OPEN CLAIMS

NI NEW
$\forall A.\ \mathrm{Sec}_A(P)$

BIND NEW
$\mathrm{Bind}(T, I)$
stays open almost to the end

Step 1. What we are actually trying to prove

Open with the goal and with the one obligation that is easy to forget: a verdict is a statement about *a particular artifact*. "This program is secure" is not a well-formed claim until you say which bytes you mean.

That second obligation is not pedantry. A confidentiality result computed on LLVM IR captured before a late pass runs, whose mutated output then proceeds to instruction selection, is a statement about a module that never executed. The honest result in that situation is UNKNOWN, and nothing weaker than a binding to the exact artifact — bitcode hash, ordered pipeline digest, the capture point, and the first IR-to-machine-IR boundary — rules it out.

Sec_A is indexed by a *coalition* A, a set of principals who may pool what they observe. Step 3 explains why the index is there. For now note that a single row in the panel stands for the whole family: the ledger convention from Part 1.5 is that an obligation quantified over an index is one row with a subscript, not one row per instance.

OPEN CLAIMS

NI USED
$\forall A.\ \mathrm{Sec}_A(P)$

BIND
$\mathrm{Bind}(T, I)$

MAN NEW
$\mathrm{Total}(\mathcal{M})$

Step 2. Declare the policy, and make the declaration total

Nothing about (1) is decidable until four questions have declared answers: which inputs are secret, who observes what, which releases are authorized and to whom, and which host executes each function.

These are supplied by a *policy manifest*, and the obligation this step opens is that the manifest is *total* over everything reachable. Partiality is the failure mode, not incorrectness: a reachable function with no declared host has no observation projection, so no verdict follows for it — and the tempting default of treating an unplaced function as local and trusted is an invented premise.

Two rules about the manifest are worth stating early because they are the ones implementations break.

Policy may not be inferred from names. Not from an LLVM value name, not from an argument position, not from `noalias`. This sounds like a style guideline and is in fact load-bearing: it is what forces the manifest to be a parsed input rather than a convention. It also has a blunt mechanical justification — `mlir-opt` discards SSA value names at parse, so a policy encoded in a name does not survive to the analysis at all.

Authority is not one relation. A single principal-to-clearance map cannot express the policies people actually want. The manifest instead carries several independent relations:

visibility	who may **observe** an item; drives the projection
audience	who a released value is **for**; drives the release premise
placement	which host **executes** a function

Visibility is itself carried by four separate bases — over hosts, components, outputs, and errors — which is the concrete reason a single clearance level per principal is insufficient: an item's observability is a function of where it lives, not only of who is asking.

Remark 3.1 (A relation the model does not yet have)**.** A natural fourth relation is *who may authorize a release*, distinct from who may read the released value. That separation is what would make "may declassify but may never read" expressible — an auditor trusted to decide *that* a summary may be published, without being trusted to see the raw data behind it.

No such relation exists in the specification. Releases are identified by policy and audience; there is no principal-level authorizer anywhere in the manifest schema. So the auditor policy above is currently **not expressible**, and the design examples that use an `authorizers` field are a proposed extension rather than a description of the system. This is a genuine gap, not a simplification for exposition.

OPEN CLAIMS

NI REDUCED
$\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$

BIND
$\mathrm{Bind}(T, I)$

MAN
$\mathrm{Total}(\mathcal{M})$

CLO DISCHARGED
$\mathcal{A} = {\downarrow}\mathcal{A}_{\max}$
immediate, and therefore easy to skip

Step 3. Derive the coalitions; do not accept the authored list

The coalition family is the *downward closure* of the authored maximal list, and it includes the empty coalition, which represents the world-visible observer. Every member must be checked, including members nobody wrote down.

This step creates its obligation and discharges it immediately, because the closure is a finite set operation. The reason to give it a step at all is that "obvious and immediate" is not the same as "done," and the failure is silent: an

implementation that iterates the authored list rather than the derived closure produces confident green results on artifacts that leak.

Two facts make the closure irreducible — neither is an implementation detail.

Verdicts are not monotone in the coalition. The natural intuition is that a larger coalition observes more, so checking the largest is the hardest case and suffices. Release audiences break this. Authorization is *per-audience*, so whether a given release retires a pair's obligation (Part 2.3) depends on which coalition is asking. For a coalition inside the declared audience the differing values are permitted and the obligation retires; for a coalition outside it, nothing retires, the obligation is still active, and *the identical operation is a leak*. So VERIFIED at one coalition and UNSAFE at another, with no containment between them.

Joint visibility is not reconstructible from singletons. An item may be declared visible to {alice, bob} jointly and to neither alone — two shares of a secret, recoverable only together. A visibility relation stored as principal-to-items cannot express this, and a report built by evaluating singletons and taking unions cannot derive it.

Together these forbid deduplicating rows by coalition monotonicity, and forbid omitting a derived coalition from the report. The cost is real and structural: the closure over n principals has 2^n members, each needing a row.

OPEN CLAIMS

- **NI** $\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$
- **BIND** $\mathrm{Bind}(T, I)$
- **MAN** $\mathrm{Total}(\mathcal{M})$
- **NF** NEW $\mathrm{NFConf}(T, I)$

Step 4. Get inside a fragment you can actually reason about

Real LLVM IR is too large a language to have an exact semantics for. The move is to fix a closed-world fragment — a whitelist of types, opcodes, constants, and intrinsics — and to define the analysis only there. Everything outside is out of profile.

The critical rule is what happens on the boundary. If an artifact does not conform, the result is UNKNOWN with a reason naming the offending construct. It is emphatically **not** a counterexample, even when the nonconforming IR contains what looks unmistakably like a secret-dependent branch. A best-effort pre-conformance scan may emit a triage finding, but that record is non-authoritative and may never be presented as a replayable witness.

The reason for that asymmetry is that a counterexample is a claim about the exact semantics, and outside the fragment there is no exact semantics to make a claim about. Reporting UNSAFE there would be unsound in the same way that reporting VERIFIED would be — just less obviously so, and therefore more dangerous, because a false alarm looks like diligence.

Remark 3.2 (A scoping fact worth internalizing)**.** The fragment is scalar: integers and pointers, plus `float` and `double`. Vectors are out of profile, as are the exotic float formats (`half`, `bfloat`, and the 80- and 128-bit forms). So an analysis defined on it says nothing about vector or tensor code until each is explicitly admitted.

Floating point is admitted but is not thereby free: a reachable arithmetic operation whose result could be an ambiguous NaN payload is a refusal, because the fragment does not choose among the payloads the IR permits. Bit-preserving movement and comparison stay eligible. Either way, a corpus of integer fixtures does not exercise these refusal paths, and a real workload triggers them immediately.

OPEN CLAIMS

- **NI** $\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$
- **BIND** $\mathrm{Bind}(T, I)$
- **MAN** $\mathrm{Total}(\mathcal{M})$
- **NF** $\mathrm{NFConf}(T, I)$
- **VAC** NEW $\mathrm{Adm}, \mathrm{PairDom}_A \neq \emptyset$
  a claim-adequacy gate, not a

Step 5. Check that the question is not vacuous

Here is a way to obtain a proof of (1) for any program whatsoever. Declare preconditions that contradict each other. There is then no admitted state, hence no admitted low-equal pair, hence the initial state set of the product is empty, hence the bad state is unreachable. The formula is unsatisfiable without executing a single instruction, and the checker reports success.

This is the countermodel that motivates the gate, and it has a subtler second form: preconditions that admit exactly one state. Now a nonempty admitted set and a nonempty pair domain both hold, via the pair of that state with itself, while no secret component actually *varies*. The product is inhabited and still proves nothing.

So two things must be exhibited, not assumed: at least one admitted execution, and at least one admitted *coupled low-equal pair* for each coalition. And a third disposition is needed for the middle case — a secret component that provably cannot vary is reported as constrained-or-unexercised rather than silently contributing to a proof, and the verifier may not infer an expected-to-vary assertion from a component's name, width, or classification.

Remark 3.3. This gate is not a logical premise of (1); it is a *claim-adequacy* gate on the report. The distinction matters for the data structure: both kinds of blocker produce UNKNOWN with a reason, but only premises appear in the antecedent of the theorem being claimed, so a report that cannot tell them apart cannot state which theorem it proved.

OPEN CLAIMS

- **NI** $\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$
- **BIND** $\mathrm{Bind}(T, I)$
- **MAN** USED $\mathrm{Total}(\mathcal{M})$
- **NF** $\mathrm{NFConf}(T, I)$
- **VAC** $\mathrm{Adm}, \mathrm{PairDom}_A \neq \emptyset$
- **DIAG** NEW DiagOK

Step 6. Run the diagnostic layer, and let it fail

Now the cheap pass from Part 2: one Low/High propagation assigning every site a disposition. Its obligation is only that the assignment is *total* — every site classified, nothing silently skipped.

The disposition domain is where the discipline lives:

not observable	no coalition-visible payload claim at this site
statically discharged	closed by a named rule, with premise references
definite violation	a violation with a witness recipe
RELATIONAL REQUIRED	precision lost; **the product must decide**
UNKNOWN	this *analysis* did not complete: a health failure

The last two rows are not two grades of the same thing, and the corpus records their conflation as a resolved defect. RELATIONAL REQUIRED is a *precision* outcome: it is this layer working exactly as designed. The diagnostic UNKNOWN is a *health* failure — incomplete traversal, a malformed record, a failed consistency check — and it is the only diagnostic disposition that blocks PROVED. Keeping them apart is what lets a run say "I was imprecise here" without also saying "I may be broken."

The last row is narrower than it looks and is worth not misreading. A *diagnostic* UNKNOWN is reserved for a failure of the diagnostic pass itself — it did not achieve its required whole-entry coverage. That is a different thing from the artifact-level UNKNOWN of Step 4 and Step 8, which reports missing semantics or an absent contract. Two layers, two UNKNOWNs, and only the second is a verdict about the program.

The fourth value is the one that makes the design work, and the one an implementer is most likely to try to eliminate. Consider four programs that are genuinely safe and that this layer cannot see are safe:

- a secret written to a slot and then fully overwritten by a public value before any read — needs a strong update, which this layer may not do;
- a secret at one byte offset with the sink fed from a disjoint offset — needs byte-exact disjointness, and the diagnostic's memory regions are not proof certificates;
- `xor` of a value with itself — needs value congruence, which a label lattice collapses away;
- a branch on a secret whose two edges target the *same* block — structurally lockstep, so disjunct 2 of Part 2 cannot fire.

All four are RELATIONAL REQUIRED here and PROVED at the product. None of them is "silence"; demanding silence is what pushes an implementer toward *statically discharged* or *not observable*, which Remark 2.6 forbids. There is even a reserved reason code for having taken that shortcut.

The fourth example carries a trap worth its own sentence. The obvious repair — "identical successors imply no control leak" — is unsound, and Step 7 exhibits the program it accepts.

OPEN CLAIMS

NI
$\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$

BIND
$\mathrm{Bind}(T, I)$

MAN
$\mathrm{Total}(\mathcal{M})$

NF USED
$\mathrm{NFConf}(T, I)$

VAC
$\mathrm{Adm}, \mathrm{PairDom}_A \neq \emptyset$

DIAG DISCHARGED
~~DiagOK~~

PROD NEW
$\mathrm{ProdSafe}_A$

Step 7. Build the exact product

Construct the two-lane product over the whole entry, with the bad-state disjunction from Part 2, and ask whether a bad state is reachable. The diagnostic layer's obligation discharges here: its job was triage, and the product now carries the burden.

Line 1 is the exception that makes the construction correct rather than merely strict. A release event does not go through the disjuncts at all; it is routed to the prefix-causal transition of Part 2.3, where an *authorized* release whose two values *differ* is permitted and retires the pair's obligation. Without that routing every authorized declassification would register as a leak, and the checker would reject every useful program.

Algorithm 1 Bad-state predicate for coalition A at aligned step k. Applies *outside* the release transition of Part 2.3; a release event is routed there instead, and an authorized release with differing values is permitted rather than bad.

1: **if** this step is a release event **then return** LEDGERSTEP
2: **if** exactly one lane can take a step **then return** BAD
3: **if** the lanes' next control locations differ **then return** BAD
4: **if** their termination or failure classes differ **then return** BAD
5: **if** the emitted event's site or occurrence index differs **then return** BAD
6: **if** $\mathrm{Proj}_A(\mathrm{event}_0) \neq \mathrm{Proj}_A(\mathrm{event}_1)$ **then return** BAD
7: **return** OK

Three places where an independent implementation plausibly diverges, and where divergence is unsound rather than merely different:

Line 3 keys on successor identity, not on the condition's label. A branch on a secret into one block has a singleton successor set and cannot fire this line, which is exactly why the fourth example in Step 6 is safe. But now consider

a branch on a secret into one block whose two *edges carry different operands* — in the LLVM dialect a ϕ is a block argument, so the merged value arrives as an argument of the target block with no operand edge from the branch. Lines 2 through 5 all pass. The leak lands only on Line 6.

That is the program the repair rejected in Step 6 would accept, and it is not hypothetical: canonicalizing an ordinary two-armed diamond in `mlir-opt` produces precisely this shape. It also refutes a more general principle — that an ordinary SSA slice is closed around a ϕ — since a dependence relation built over operand edges never sees the gating fact.

Line 5 is global, not per-site. Comparing occurrence counts *per event template* is not the same as comparing the global interleaving. Two lanes can emit the same events, the same number of times, with identical payloads, in the opposite order. Every per-template row agrees; the traces differ. So the projected trace must be an ordered sequence with pairwise index alignment and a first-divergence position — never a multiset, and never a map from site to last value.

Lines 2–5 do not weaken under projection. They are structural, so a coalition that cannot read a payload still distinguishes these executions. An implementation that applies the coalition projection before evaluating them will wrongly equate lanes that differ in control location, count, or termination.

Remark 3.4 (What may not be filtered)**.** An execution that exhausts an unrolling bound, faults, or risks undefined behaviour is *retained*, not deleted. Deleting it silently narrows the proof domain — the same vacuity failure as Step 5 arriving by a different route. Bound adequacy is a universal obligation discharged separately, not a filter applied inside the admitted set.

Step 8. Prove, refute, or refuse

The product query returns one of three things, and the third is not a failure of the tool.

Unreachable ⇒ PROVED, provided the adequacy gate of Step 5 holds. That proviso is why both obligations discharge in the same step.

Reachable, with a witness ⇒ COUNTEREXAMPLE, but only if the witness *replays*. A model returned by a solver is a candidate, not a finding: it must be re-executed under the exact semantics and shown to reach the bad state, with its prefix covered. Failed replay or missing coverage is UNKNOWN and tool inconsistency — never a counterexample. This is the obligation that opens and closes inside this step.

Neither ⇒ UNKNOWN, with the blocker set named.

OPEN CLAIMS

- **NI** $\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$
- **BIND** $\mathrm{Bind}(T, I)$
- **MAN** $\mathrm{Total}(\mathcal{M})$
- **NF** $\mathrm{NFConf}(T, I)$
- **VAC** DISCHARGED ~~$\mathrm{Adm}, \mathrm{PairDom}_A \neq \emptyset$~~
- **PROD** DISCHARGED ~~$\mathrm{ProdSafe}_A$~~
- **WIT** DISCHARGED ~~$\mathrm{Replay}_A(w)$~~

Two rules about aggregation, both of which are easy to get wrong and neither of which is cosmetic.

A replayed witness outranks an open universal obligation. If a witness reaches a bad state and replays, the verdict is COUNTEREXAMPLE regardless of an unfinished bound-adequacy proof, an unfinished definedness proof, or another solver timeout elsewhere — and regardless of whether a solver, a fuzzer, a differential search, or a person found it.

Source-independence is a requirement on the *verdict*, not on the record. The result must be identical from every discovery route, and no step of the argument may consult provenance. The record itself is the opposite of anonymous: it must **retain** the discovery source and the seed or model, precisely so the candidate can be re-executed and the replay audited. Confusing the two — stripping provenance in the name of source-independence — destroys reproducibility while buying nothing, since it is the argument that must ignore the source, not the file.

Otherwise every blocker is preserved. With one blocker the result is UNKNOWN carrying it. With several it is UNKNOWN carrying a compound reason over the *complete ordered* blocker set. A flat enumeration of reason codes cannot represent that, which makes the compound constructor a genuine requirement on the record type rather than a convenience.

Remark 3.5. Note the shape of the record this forces. UNSAFE carries a replayable witness; UNKNOWN carries an ordered blocker set; PROVED carries neither. A single "reason" string field cannot hold all three, and a design that tries will end up attaching reason codes to proofs, which are reason-free.

OPEN CLAIMS

NI
$\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$

BIND
$\mathrm{Bind}(T, I)$

MAN
$\mathrm{Total}(\mathcal{M})$

NF
$\mathrm{NFConf}(T, I)$

DEP NEW
$\mathrm{PairRef}_A$
leaving this open is what CONDITIONAL *means*

Step 9. Close the deployment gap, or admit it is open

Everything so far concerns a frozen artifact under a declared machine model. Two gaps remain between that and a running system, and they are of different kinds.

L3 is *attribution*: relating source, imported IR, and the final target artifact, so a violation or a repair can be ascribed to a specific compiler boundary. This is where the classic "the compiler broke my constant-time code" findings live — a conditional move lowered to a branch on a target without conditional moves, or a division strength-reduced into a multiply-high so the operation you were reasoning about is not the operation that executes.

L4 is *facts outside the modeled semantics*: real helper latency, compressor behaviour, hardware isolation between tenants, sanitizer sufficiency, certificate soundness, and whether the backend preserved the observable trace at all. These are not provable from the IR. They are empirical or hardware facts.

The honest handling is the fourth outcome. A repair that is correct in the model but depends on an undischarged external fact is CONDITIONAL, and must name every assumption it rests on. An UNSAFE or VERIFIED result may name none.

Remark 3.6 (Why this obligation is per-coalition and separate)**.** Deployment status is an independent axis from model status. A model-level counterexample cannot be repaired by deployment evidence, and deployment evidence cannot upgrade an UNKNOWN model result. Collapsing the two into one verdict token is the most common structural error, and it shows up concretely: an obligation

like “the backend preserved the trace” and one like “this helper’s latency is operand-independent” belong to different axes and should not share a field.

Step 10. Close the arc

Discharge the artifact-level gates and the goal. The manifest was total, the artifact conformed, the results are bound to those exact bytes and that exact pipeline, and the deployment obligations of Step 9 were *discharged*. The product was unreachable under a non-vacuous admitted domain, so (1) holds for every derived coalition at the claimed boundary.

This is the VERIFIED branch, and it is the only branch in which the arc closes. Had the deployment obligations merely been *named* rather than discharged, the outcome would be CONDITIONAL — and the ledger would end with that row still open, which is exactly the invariant from Part 1: VERIFIED may carry no obligations, so a non-empty ledger is not a proof but a different verdict. An empty ledger is the picture of one specific outcome, not of the analysis having run.

What has *not* been established is worth saying explicitly, because it is the difference between an honest result and an overclaim:

- nothing about constructs outside the fragment of Step 4;
- nothing about a different artifact, pipeline, or target profile;
- nothing about real latency, hardware isolation, or speculation beyond what Step 9 explicitly named and discharged;
- nothing about coalitions of principals absent from the manifest, since the closure was taken over the declared set.

Every one of those is a boundary, not a bug. The value of the four-outcome contract is that it can state them.

The ledger is empty. Most middle steps traded an obligation for a successor that was easier to attack; a few only opened one, because some preconditions have to be stated before anything can be traded against them. The two that stayed open longest were the least glamorous: that the policy was total, and that the answer was about the artifact we actually shipped.

OPEN CLAIMS

- **NI** DISCHARGED ~~$\forall A \in \mathcal{A}.\ \mathrm{Sec}_A$~~
- **BIND** DISCHARGED ~~$\mathrm{Bind}(T, I)$~~
- **MAN** DISCHARGED ~~$\mathrm{Total}(\mathcal{M})$~~
- **NF** DISCHARGED ~~$\mathrm{NFConf}(T, I)$~~
- **DEP** DISCHARGED ~~$\mathrm{PairRef}_A$~~

OPEN CLAIMS

nothing left to prove

4 What the harness actually establishes

Part 3 describes a checker that does not exist yet. What exists is a regression corpus that pins the obligations the checker will have to discharge. This part explains what that corpus does and — more usefully — what it does not.

4.1 Two kinds of test

The suite contains 45 IR fixtures and 7 integration tests, and they split into two categories that behave completely differently.

Green tests are facts locked in. They assert something true right now about the IR, the metadata, or the toolchain. They catch regressions.

Red tests are executable specifications. Seventeen fixtures fail on purpose. Each carries a diagnostic run with an expected diagnostic naming the exact stable reason a future analysis must emit at a specific operation, and each fails with *expected error not produced*. They are bring-up gates, not disabled tests: implementing the analysis and inserting it into those runs is what turns them

green. A suite where every test passes would be a suite that had stopped specifying anything.

4.2 Four mechanical layers

Layer	What it establishes
IR validity	every fixture parses and satisfies dialect invariants
Shape	the decisive operation — store, branch, address, division, allocation, backedge — is present and connected
Shape stability	for the eight fixtures carrying a `-canonicalize` run — the four precision controls, the allocation pair, the loop, and the predecessor-choice anti-control — the decisive shape survives canonicalization, so those cannot silently decay into a different scenario. The other 37 pin shape only in the form they were written
Metadata	each fixture has exactly one of the four outcomes, one observer, a stable reason, obligations required precisely when the outcome is UNKNOWN or CONDITIONAL, and real provenance

Note what is absent. **No layer checks an information-flow result**, because no analysis is wired in. The corpus states the specification; it does not yet test an implementation of it. Being clear about this is the difference between a useful artifact and a misleading one.

4.3 Controls, and why every control needs an anti-control

A corpus of leaky programs is satisfied by an analysis that reports every program as unsafe. That is not a hypothetical failure — it is the cheapest way to pass a suite built only from violations, and it makes the tool worthless without making any test go red.

The guard is a family of *precision controls*: fixtures that are genuinely non-interferent, where a reported violation is imprecision rather than a finding. The four in the corpus are exactly the four examples from Step 6: identical-successor branch, value cancellation, public overwrite before observation, and offset-disjoint reload.

But controls alone are unsafe to ship, because the cheapest way to quiet a control is usually an unsound repair. So a control wants a paired **anti-control** that must retain the opposite outcome:

Control	Unsound repair its partner blocks
identical-successor branch	"identical successors imply no control leak" — the partner is the same single-successor branch with differing block-argument operands
public overwrite	adding a proof-authoritative strong update to the diagnostic layer
offset-disjoint reload	coarsening offsets to a cache line, which is a refusal and never a proof
public world-structural allocation size	refusing every dynamic allocation, or treating an equal public cap as an equal size

The rule is: *never satisfy one member of a pair by weakening the other.* The first row is the sharpest instance, and it is not a stylistic worry. Running `mlir-opt`'s canonicalizer on an ordinary two-armed diamond produces a single-successor branch whose edges carry different operands — byte-identical in shape to the control, differing only in the operands. An implementation repaired by that rule accepts a real leak, silently.

4.4 Which step each fixture guards

Step	Fixtures
Step 2	the wrong-party and wrong-host families; the actor examples for the four relations
Step 3	the downward-closure example, where the leak exists only at a coalition nobody authored
Step 4	currently *uncovered*: no fixture exercises a non-conformance refusal
Step 5	currently *uncovered*: the vacuity countermodel is not encoded
Step 6	the four precision controls
Step 7	the predecessor-choice anti-control; trace order and multiplicity are *not* yet encoded
Step 8	the unproved-alias and bound-exhaustion refusals
Step 9	the CKKS and wolfSSL helper-latency families, the only CONDITIONAL fixtures

4.5 One call, four outcomes

The wolfSSL 3579 family is the most instructive group in the corpus, because it demonstrates that the observer model is a load-bearing *input* rather than a label. *Three* fixtures share one byte-identical body — a wide multiply lowered to `@__muldi3` on a 32-bit target — and differ only in the declared helper profile. Two more complete the family without sharing that body: the un-lowered source, whose whole body is a single `llvm.mul`, and a repaired constant-time replacement, which is a 64-iteration mask/add loop that emits no helper call at all:

Variant	Outcome	Declared knowledge
source	UNKNOWN	no target lowering, no helper contract
target, no contract	UNKNOWN	the call exists; no latency summary
target, affected profile	UNSAFE	a named operand-dependent helper profile
target, constant latency	VERIFIED	a named constant-latency profile
target, repaired	CONDITIONAL	repaired, pending target evidence

Three of the four outcome values — UNKNOWN, UNSAFE, VERIFIED — on one byte-identical body, driven entirely by declarations. The CONDITIONAL row is different in kind: it comes from a code repair, not a declaration change, and saying otherwise would overstate an already strong point. That is the cleanest demonstration available of the point in Step 2: `L0` is a contract, not proof of the contract, and the same instruction is UNKNOWN, UNSAFE, or VERIFIED according to what has been declared about the machine it runs on. Note also that the two UNKNOWN rows are genuinely different scenarios with different missing facts, not one file with an undecided verdict.

4.6 Countermodels as regression tests

Four of the metatheory's required negative results are encoded. These are the tests a *re-implementation* would fail: they capture reasoning errors rather than coding errors, and reasoning errors are the ones that survive review.

Fixture	Outcome	Invalid principle refuted
predecessor choice	UNSAFE	an SSA slice is closed around a ϕ
prefix-causal release	UNSAFE	a future release may condition an earlier observation
alias unproved	UNKNOWN	unproved ABI separation may be assumed
bound exhausted	UNKNOWN	bounded-run filtering is a sound proof domain

The two UNKNOWN rows deserve their outcome explained, because UNKNOWN reads as a weaker result than it is. Unproved aliasing yields neither safety nor a counterexample: with no proved disjointness clause and no declared may-alias realization in the product, asserting UNSAFE would itself be unsound, since a counterexample requires a witness that replays under the exact semantics. Likewise a reachable bound-exhausted transition that yields no replayed bad execution is UNKNOWN with the bound-adequacy obligation open — promoting it to a counterexample is exactly what Step 8 forbids.

The bound-exhaustion fixture is the corpus's only loop with a *secret-dependent* trip count. Two earlier fixtures already contain fixed-bound loops with loop-carried

block arguments — `secret_embedding_index.fixed`, a 16-iteration public-induction table scan, and `wolfssl_3579_mul.target_fixed`, a 64-iteration mask/add multiply — so backedge joins were exercised before it. What was absent is a backedge whose iteration count depends on a secret, and therefore any exercise of bound adequacy at all.

4.7 The honest gaps

The outcome distribution is VERIFIED 21, UNSAFE 17, UNKNOWN 5, CONDITIONAL 2. For a tool whose central claim is graceful degradation, five refusals is thin: there is no fixture for an indirect call, recursion, inline assembly, a volatile or atomic access, an uninitialized read, or ambiguous floating-point. Each of those is a place the analysis will meet something it cannot model, and an untested refusal path is where a silent VERIFIED comes from.

Three further gaps are worth naming precisely:

- About twenty of the older fixtures encode their entire safe/unsafe distinction in SSA value names, which `mlir-opt` discards at parse. After parsing, such a fixture is two indistinguishable stores to two unannotated pointers — so its bring-up gate is not merely unimplemented, it is unsatisfiable in principle. The newer fixtures carry policy in discardable attributes, which do survive parsing and canonicalization.
- The corpus is entirely scalar, with three loops, only one of which has a non-constant trip count, so nothing exercises fixpoint termination at scale, and nothing exercises the vector, floating-point, or tensor refusals that any real workload triggers immediately.
- The precision controls carry no diagnostic run, deliberately: their correct disposition is RELATIONAL REQUIRED, and a zero-directive run would encode the wrong requirement. So they pin shape today and become live precision gates only once the record can name an expected disposition. Concretely, the suite can currently catch an analysis that *misses* a leak but not one that *invents* one.

4.8 The gap the corpus does not know it has

Every fixture above describes a leak that a *programmer* wrote. Twenty reproduced compiler transformations (Part 6) describe leaks that a *compiler* introduced, and the corpus has essentially no coverage of them.

The distinction is not academic. A programmer-authored leak is present in the source, so source review can find it and the fixture merely regression-tests the tool. A compiler-introduced leak is absent from the source *and* absent from the analysed IR — no review of either finds it, and the tool is the only thing that could.

Worst of all is the direction with zero coverage:

$$\text{leaky source} \longrightarrow \textbf{clean -O2 IR} \longrightarrow \text{leaky binary}$$

Mid-end speculation turns a secret branch into a select, which reads as a repair. The backend's cmov-to-branch conversion turns it back into a branch. An IR-level checker sees the middle stage and reports VERIFIED. That is not a missing fixture;

it is a missing *axis* — nothing in the corpus asserts anything about a level below the one it analyses.

Three further findings show the corpus can be actively misleading here:

- **The compiler undoes the corpus's own repairs.** Giving each tenant a separately scoped buffer is the natural fix for the LeftoverLocals pair. Stack colouring merges the two allocations back onto one frame offset, with runtime-confirmed residue — and the lifetime markers that prove temporal separation are exactly what authorises the merge.
- **A policy gate can be deleted outright.** A measured fail-closed-to-fail-open regression: the same program traps at `-O0` and, at `-O2` with the enforcement slot optimised away, performs the release and exits successfully.
- **Metadata is discarded on merge.** When two arms are sunk into one, unknown metadata kinds are dropped rather than intersected, leaving a secret store to a public pointer carrying no label at all. Under the convention that unlabelled means public, that is a false negative manufactured by a routine optimisation.

Closing this gap does not mean writing twenty more fixtures at the same level. It means the corpus must be able to state a fact about *two* levels at once, which is what Part 6 is about.

5 How would you know the checker is correct?

"Is it correct?" is two questions wearing one coat, and separating them is most of the work.

1. **Is the specification right?** Does (1) capture what you mean by confidentiality? This is *not* provable. It is a definition.

2. **Is the implementation right with respect to the specification?** This is provable, in layers, at wildly varying cost.

Conflating them produces the characteristic formal-methods overclaim: a mechanized proof of question 2 presented as if it settled question 1.

5.1 Validating the specification: countermodels, not proofs

You cannot prove a definition. What you can do is show that plausible *alternative* definitions are wrong, which is what a countermodel is for: a small executable program witnessing that some appealing principle fails.

The metatheory names seven such required negative results, and they are worth reading as a validation *method* rather than a list. Each is a principle somebody would naturally adopt, plus a program refuting it:

Appealing but invalid principle	Refuted by
bounded-run filtering is a sound proof domain	a secret selecting a trip count past the bound
a future release may condition an earlier observation	emit-then-release, two operations
local per-template checks determine global trace order	two events, same payloads, opposite order
an ordinary SSA slice is closed around a ϕ	the predecessor-choice diamond
unproved ABI alias separation may be assumed	store through p, load through q, $p = q$ admitted
an empty premise domain supports a meaningful proof	contradictory preconditions
unary refinement preserves confidentiality	a refinement safe alone, unsafe when paired

Four of these are encoded in the harness today. The value of encoding them is that they are the tests a *re-implementation* would fail — they capture the reasoning errors, not the coding errors, and reasoning errors are the ones that survive code review.

Remark 5.1. This is the strongest available answer to "how do I know the property is the right property." It is an argument from the absence of known counterexamples to the definition, strengthened by having actively searched for them. That is weaker than a proof and much stronger than an assertion.

5.2 A ladder of assurance for the implementation

Order these by cost per unit of confidence, cheapest first. The interesting observation is that the cheapest three are rarely done, and they subsume most of what people hope to get from the expensive one.

Rung 1: metamorphic and differential testing

Properties of the checker itself, requiring no ground truth:

- **determinism** — two runs on one input produce byte-identical reports. Worth a deliberate worklist-shuffling switch so the guarantee is falsifiable rather than accidental; every pointer-keyed hash map on the report path is a hazard here.
- **permutation invariance** — renaming SSA values or reordering independent operations must not change the verdict. Note this splits in two: the *verdict* must be invariant, while the serialized blocker sequence must be *canonical*. A single test comparing whole reports conflates them.
- **monotone refusal** — obfuscating the input may turn VERIFIED into UNKNOWN, never the reverse.
- **idempotence** — re-running on the checker's own normalized output agrees.

Rung 2: an exhaustive small-domain oracle

This is the highest-value step available and the one most often skipped.

For *small* bit-widths the property is decidable by brute force. Take a generated program over, say, 4-bit integers and a handful of memory cells. Enumerate every pair of secret inputs whose entry states are coupled — equal public configuration, equal public alias topology, equal initially-visible components — run both lanes concretely, and compare the projected traces step by step, routing release events through the prefix-causal transition of Part 2.3. That yields **ground truth** — not an approximation — for that program.

Remark 5.2 (The oracle must not use the whole-run encoding either)**.** It is tempting to write the enumeration the short way: keep only the secret pairs whose end-of-run authorized releases agree, then compare traces. That is Remark 2.4 all over again, and it makes the oracle agree with a buggy checker rather than catch it — including on the harness's own `prefix_causal_release.bad.mlir`. An oracle built on the refuted encoding validates nothing. Couple at entry; retire obligations as releases occur.

Now you have a soundness oracle, and you can fuzz against it:

- generate random programs in the fragment plus random manifests;
- compute the brute-force verdict and the checker's verdict;
- any program where brute force finds a leak while the checker reports VERIFIED is **a soundness bug**, reduced automatically to a minimal fixture;
- the reverse direction is not a bug but a precision measurement, and tracking its rate over time is the only honest way to report precision.

Two things make this unusually effective here. First, the analyzable fragment is a *closed-world whitelist*, so a generator covering it is a finite, enumerable engineering task rather than an open-ended one. Second, every soundness bug it finds arrives as a concrete, minimal, checked-in regression fixture — exactly the artifact the corpus wants. A mechanized proof, by contrast, tells you a theorem holds but hands you no fixture.

Rung 2b: cross-level differential

Rung 2 fuzzes one level against ground truth. The cheaper sibling fuzzes one level against *another level of the same program*: analyse before and after each lowering and compare verdicts. No oracle is needed, because the two verdicts are each other's oracle.

VERIFIED above and UNSAFE below is laundering — the lowering introduced the leak. The reverse is evidence destruction. Degradation to UNKNOWN is fine. This is the check that catches the entire class in Part 6, it reuses the corpus unchanged, and it is the reason that part argues for extraction at several levels rather than one.

Rung 3: certificate-producing analysis

Rather than proving the analyzer correct, have it emit a *certificate* that an independent, much smaller checker validates: an unsatisfiability proof from the solver, the product encoding, and the mapping from IR sites to product constraints.

The payoff is a shrunken trusted computing base. You stop needing to trust a large, fast-changing analysis and start needing to trust a small checker plus the

semantics. This is the same trade that makes proof-carrying code and validated translation attractive, and it has a decisive practical advantage: the analysis can be rewritten freely without invalidating anything, because each run re-establishes its own validity.

Rung 4: mechanized proof of the core

Worth doing — for a carefully chosen core, discussed next.

5.3 Mechanizing it for MLIR: what is and is not well-posed

Start with the uncomfortable part. **"Prove this correct for MLIR" is not a well-posed goal.** MLIR is a meta-IR: an extensible framework whose dialects are open and user-defined. There is no fixed language to have a semantics for, so there is nothing to quantify over. Any mechanization must first fix a dialect subset and give *that* a semantics.

The good news is that this project has already done that, without framing it as a prerequisite for proof. The normal form of Step 4 *is* the fixed subset: a closed-world whitelist of types, opcodes, constants, and intrinsics. That is the object to mechanize.

Remark 5.3 (Reusable prior work, checked)**.** The foundation is in better shape than the framing above suggests, and it points at LLVM-IR level rather than MLIR level.

Vellvm is a mechanized semantics for a large sequential subset of LLVM IR, actively maintained against Rocq, and since its 2021 rewrite it is structured around interaction trees rather than the original 2012 relational semantics. Decisively for this programme, *noninterference has already been given a semantics in the same interaction-tree idiom*, by overlapping authors. That means the two halves — an LLVM semantics and a noninterference semantics — exist in one compatible formalism, which is the single largest reuse opportunity available and a strong argument for expressing the fragment at LLVM-IR level.

Alive2 does bounded translation validation of LLVM optimizations via SMT. Not a proof, but exactly the right shape for validating the normalizer, per milestone 6 of the ladder below. ("Milestone" rather than "step" throughout this part: *Step* is reserved for the numbered steps of Part 3.) *CompCert* proves functional correctness of a C compiler: architecturally instructive, different property. Its constant-time-preserving successor is the closer analogue and is discussed in Part 5.4.

For MLIR specifically, mechanized semantics exist in Lean and, separately, as dialects carrying semantics lowered to SMT — the latter has already found miscompilations upstream. Neither carries an information-flow theory.

The decomposition that makes it tractable

Split the goal three ways and give each part the treatment it can actually support:

Part	Content	Treatment
(a)	(1) is the right property	definition; reviewed and countermodel-validated, *not* proved
(b)	the product construction implies the property	mechanized once; this is the core theorem
(c)	this run's artifacts are valid	per-run certificate checking, not proof

Almost all the leverage is in refusing to prove (c). The analysis, the normalizer, and the solver integration will change constantly; proving them once means re-proving them forever. Validating each run costs a fixed engineering investment and survives arbitrary rewrites.

The core theorem, stated

The load-bearing obligation is:

Theorem 5.4 (Product soundness)**.** *For every artifact T in the fragment, every entry e, and every coalition A: if no bad state of the two-lane product is reachable from the coupled initial states, then* Sec_A *holds at e — that is,* $\mathrm{ProdSafe}_A(T, e) \Rightarrow \mathrm{Sec}_{A,e}(T)$.

Everything else the checker does is a strategy for deciding the antecedent. This direction is already proved: it is Theorem R8.3 of the metatheory, with the full premise chain assembled in R9.1 and the solver form in R9.2 — an accepted UNSAT for the identity-bound bad-reachability formula establishes Sec, and UNKNOWN does not.

The completeness direction is available, and unstated

It is tempting to say that only soundness is needed and that completeness is deliberately abandoned in exchange for the right to answer UNKNOWN. That is not quite the situation, and the difference is worth a theorem.

The metatheory's Theorem R6.3, *reachable-bad correspondence*, is a **biconditional**: a bad state is reachable in the product *if and only if* there exist admitted states and coupled choices, satisfying low-equivalence, whose unary executions have a first mismatch forbidden by the observation relation. The $\Leftarrow$ direction is the soundness of Theorem 5.4. The $\Rightarrow$ direction, contraposed, is precisely $\mathrm{Sec}_A \Rightarrow \mathrm{ProdSafe}_A$ — the completeness direction — and the corpus never states it as a theorem, corollary, or remark.

One qualifier is load-bearing rather than decorative. R6.3 quantifies over a *first mismatch forbidden by the active prefix-causal relation*, not over pairs pre-filtered by whole-run release equality. So the corollary below holds for Sec_A read prefix-causally, as constructed in Part 2.3, and is **false** under the naive whole-run reading of (1): the `emit-then-release` program satisfies the whole-run biconditional while leaking. Stating the completeness result without that qualifier would convert a correct theorem into an unsound one.

Theorem 5.5 (Relative completeness, harvestable from R6.3)**.** *For an artifact in the fragment, with every contract present, within the declared execution bound, and relative to the fixed observation relation:* $\mathrm{ProdSafe}_A \Leftrightarrow \mathrm{Sec}_A$.

Read that carefully, because the qualifiers carry it. It says the product is *exact* within the bound: no false negatives, and — the part a type system can never claim — no false positives either. It does *not* say the observation relation captures every physical channel; the metatheory is explicit that completeness here is relative to the normal-form grammar and the declared observation relation, and does not establish that the relation contains every physical timing, microarchitectural, operating-system, or network observation.

This is the sharpest available statement of what the architecture buys over a flow-*in*sensitive type system, which carries one fixed level per variable and therefore rejects programs as simple as "assign a secret, then overwrite it with a constant, then publish." (A flow-sensitive system, with a per-program-point environment, accepts that particular program — it is their motivating example. What no type system of either kind gets is the converse direction below.) It costs no new proof — only the discipline of stating a corollary that already follows.

Where completeness actually leaks

Every UNKNOWN the checker can return corresponds to a distinct formal reason the biconditional's antecedent fails to be decidable:

Formal cause	Reason-code family
undecidability of reachability, so a bound is required	bound exhausted, resource limit
fragment membership: no semantics outside the whitelist	unsupported opcode, type, intrinsic, residual vector
missing *relational* contract for a callee	missing contract, indirect call, recursion
solver incompleteness	solver timeout
the semantics is a *set* of behaviours	freeze may choose, ambiguous NaN payload, uninitialized load
memory abstraction cannot decide aliasing	alias binding mismatch, layout-dependent pointer comparison
channel absent from the observation relation	unsupported address observation profile

So the reason-code enumeration is not an error taxonomy. It is the exception list of Theorem 5.5, which has a design consequence: the list must be *closed*, and each member owes an obligation of the form *if this cause does not apply, the checker decides.*

Remark 5.6 (The half that is missing)**.** The enumeration covers *refusals.* It does not cover *conservative rejections* — cases where the analysis reports a problem on a program that satisfies the property. At least three such sources exist and none has a code: over-approximated may-alias, an observation relation strictly stronger than output-only noninterference, and the requirement of world-level constant

structure even at hosts no coalition can see. Because they carry no code, a false alarm from any of them is indistinguishable in the report from a genuine finding. If the completeness bookkeeping is to be as disciplined as the soundness bookkeeping, these need first-class representation — the same argument that produced the precision controls of Part 4.3, arriving from the theory side rather than the testing side.

A second, smaller theorem is worth mechanizing because it is where the architecture could quietly go wrong:

Theorem 5.7 (The diagnostic layer only narrows)**.** *No diagnostic disposition implies* Sec_A*, and no diagnostic disposition removes a product constraint. In particular, a relational-required disposition implies nothing whatever.*

Theorem 5.7 is small, and it is exactly the invariant that Remark 2.6 says implementations break under pressure to reduce false positives.

A milestone ladder

1. **Mechanize the fragment's operational semantics**, with the observation projection as an explicit output rather than an afterthought. Deliverable: a step relation and a trace projection. The projection being explicit is what makes the property statable at all.

2. **State** (1) **and the product formally**, and prove Theorem 5.4. This is the single highest-value proof in the programme, and it is independent of any implementation.

3. **Prove Theorem 5.7** for the actual disposition domain.

4. **Extract or validate the product encoder.** Either extract a verified encoder from the proof, or keep the fast one and check its output against the mechanized semantics per run. Prefer the latter.

5. **Do not mechanize the solver.** Consume unsatisfiability certificates from it and check those. Verifying an SMT solver is a research programme in its own right and buys little here.

6. **Validate the normalizer per run**, translation-validation style: show the normalized module is observationally equivalent to its input on this input, rather than proving the normalizer once. It will change constantly; the proof would not survive.

7. **Leave the backend to bounded validation.** Comparing the emitted machine code's observable trace structure against the IR-level one, on the actual artifact, is the only thing that addresses the trace-preservation obligation of Step 9. A verified backend would be better and is a far larger undertaking.

What a proof can never give you

Every L4 obligation is outside the reach of any proof about the IR: real helper latency, whether the backend preserved the trace, hardware isolation between

tenants, speculative execution, sanitizer sufficiency, certificate soundness. A mechanized theorem is always *relative to a declared machine model*, and the gap between that model and the silicon is exactly what the CONDITIONAL outcome exists to record.

This is worth stating loudly because it is the most common way formal claims mislead. "Proved noninterferent" with an unstated machine model is not a stronger claim than "CONDITIONAL, pending these four named facts" — it is the same claim with the assumptions hidden. The four-outcome contract is, in that light, less a limitation of this system than an unusually honest interface.

5.4 Where this sits in the literature

Worth knowing precisely, because two things here are already owned and two are not.

The product construction is classical. Self-composition as a route to noninterference is Barthe, D'Argenio and Rezk (CSFW 2004, extended in *MSCS* 2011); the framing of secure information flow as a safety problem is Terauchi and Aiken (SAS 2005); the general theory of properties over sets of traces is Clarkson and Schneider's hyperproperties (CSF 2008, *JCS* 2010); and relational Hoare logic is Benton (POPL 2004). For the type-system side of the two-layer split, the canonical soundness result is Volpano, Irvine and Smith (*JCS* 4(2–3):167–187, 1996), surveyed by Sabelfeld and Myers (*IEEE JSAC* 2003).

Constant-time verification at LLVM level is occupied. *ct-verif* (Almeida, Barbosa, Barthe, Dupressoir, Emmi; USENIX Security 2016) is the direct ancestor: product construction, LLVM, annotations for public inputs. Two facts about it matter. Its toolchain is effectively abandoned — last commit 2017 — so it cannot be run as a baseline. But its benchmark corpus survives and is still the field's de-facto suite: twelve directories covering `curve25519-donna`, `openssl`, `mee-cbc`, `polarssl`, `sodium`, `s2n` and others. **Reuse the corpus as fixtures; do not expect to reuse the tool.** The live successor is *CT-LLVM* (Zhang and Barthe, 2025), which is analysis-based — def-use plus alias analysis — rather than product-based, so it makes a different soundness/precision trade.

Compiler preservation is occupied. The mechanized analogue of the deployment obligation in Step 9 is CompCert-CT, "Formal verification of a constant-time preserving C compiler" (Barthe, Blazy, Grégoire, Hutin, Laporte, Pichardie, Trieu; POPL 2020). The optimization-level statement is Rosemann, Hack and Garg, "Non-interference Preserving Optimising Compilation" (OOPSLA 2025), which develops hyperproperty simulations — close kin to the paired-refinement notion this system uses. The general framework for what such a theorem should even say is Abate et al., "Journey Beyond Full Abstraction" (CSF 2019).

What is not occupied. Two gaps, and the second is larger.

First, a *mechanized, product-construction* noninterference checker at LLVM IR with a preservation story. The pieces exist separately — Vellvm for the semantics,

the interaction-tree noninterference work for the property, CompCert-CT for preservation — and nobody has assembled them.

Second, and more striking: **MLIR has no information-flow story at all.** There is no security, information-flow, taint, secrecy, or constant-time dialect upstream. The nearest neighbour by name, HEIR's `secret` dialect, is a lifting mechanism for homomorphic encryption; its own documentation states that `secret.generic` does not itself do anything, and it carries no noninterference property, no declassification, and no theorem. Mechanized MLIR semantics exist, and information flow has not been built on them.

That is the opening, and it argues for a specific division of labour: prove the core at LLVM-IR level where the semantics already exists, and treat MLIR as the place where the *policy surface* and the tensor-level channels live.

5.5 If you only do three things

1. Build the **exhaustive small-domain oracle** and fuzz against it. It finds real soundness bugs now, it produces regression fixtures as output, and it requires no proof assistant.

2. Make the checker **emit a certificate** and write the small independent validator. This shrinks what you have to trust, and it survives every future rewrite of the analysis.

3. Mechanize **Theorem 5.4 only**. One theorem, no implementation dependencies, and it is the statement everything else is a strategy for.

Rungs 1–3 are weeks to months. A mechanized Theorem 5.4 over a small fragment is months. A verified end-to-end pipeline is years, and would still terminate in the same `L4` obligations you started with.

6 A checker that survives lowering

Everything up to here analyses *one* artifact. This part asks what happens when that artifact is compiled further, and derives the architecture the measurements force. The short version: single-point analysis is not salvageable, and the fix is to stop pushing policy downward.

6.1 What the measurements found

Twenty semantics-preserving transformations were reproduced against LLVM 17.0.6. They sort into three gaps, and it is worth naming them separately because they fail in different directions.

The level gap. The transformation happens *below* LLVM IR, so an IR-level verdict describes a program that does not run. The sharpest instance: `x86-cmov-converter` turns a conditional move back into a branch. Composed with mid-end speculation, which turns a branch into a select, the pipeline produces the worst possible sequence — **leaky source, clean `-O2` IR, leaky binary**. The same IR yields opposite verdicts on x86-64, aarch64-generic, aarch64-neoverse-n2,

and riscv64. Related: `StackColoring` merges two lifetime-bracketed allocations onto one frame offset, with runtime-confirmed residue.

The annotation gap. The policy does not survive. `combineMetadata` discards custom metadata when merging; every signature-changing pass drops parameter attributes; `mlir-translate` drops MLIR attributes entirely, with no diagnostic. Combined with the convention that an unlabelled value is public, each of these is a false negative rather than a refusal.

The counting gap. Occurrence cardinality moves in *both* directions under plain `-O2` — sinking merges two sites into one, jump threading splits one into two. So no exact-count policy is an artifact invariant.

Remark 6.1 (The finding underneath all twenty)**.** The artifact the checker reads is not the artifact that runs, and nothing in the system says which one is normative. Every other problem in this part is a consequence.

6.2 Why single-point analysis cannot be repaired

There are only two places to put a single analysis, and the measurements close both.

Analyse early, where the policy is meaningful and the structure is intact. Then the level gap applies in full: the binary can leak through a channel that does not exist at that level, and the laundering sequence above produces a clean verdict on a leaky program.

Analyse late, close to the machine, where the observations are real. Then the annotation gap applies: the policy is gone. Worse, it is gone *silently*, so the analysis does not refuse — it sees unlabelled code and, under the usual convention, calls it public.

The obvious repair is to carry the policy downward so that late analysis has something to read. Part 6.3 shows every known carrier has a silent-loss path. That is what makes this an architectural problem rather than an engineering one.

6.3 Every carrier leaks

Carrier	Measured failure
MLIR `sps.*` attributes	`mlir-translate -mlir-to-llvmir` drops all of them; zero survive, no diagnostic
LLVM metadata	`combineMetadata` discards unknown kinds when merging two sites
Parameter attributes	dropped by every pass that rewrites a signature
Opaque marker call	must have no visible definition to survive, but then it does not link; once the definition is visible it is inlined away
Debug info	explicitly non-semantic; passes may drop or coarsen it

Note what these have in common. They are all attempts to make a *policy fact* travel alongside the code, through a pipeline whose passes have no obligation to

preserve anything except semantics. The pipeline is not misbehaving; it was never told these facts mattered.

6.4 The inversion

If policy cannot travel down, send observations up.

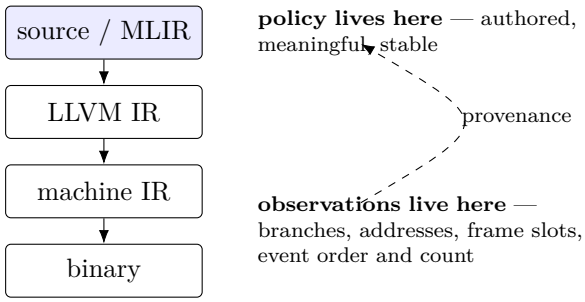

Figure 2: Policy stays at the level where it is authored. Observations are extracted at the level where they are real. A provenance map relates the second to the first, and the relational check runs once, at the top, over observations sourced from the bottom.

The three gaps close for structural reasons rather than by better engineering:

- **Level gap** — observations are gathered where they are real, so a branch introduced by the backend is simply present in the observation set.
- **Annotation gap** — labels never travel, so nothing can drop them.
- **Counting gap** — cardinality is *measured* at the bottom rather than asserted at the top, so a merged or duplicated site is observed, not assumed away.

6.5 The quantifier error, and universal discharge

The architecture as first stated has a defect worth dwelling on, because it is a *logic* error rather than an engineering gap, and because the first draft of this section contained it.

That draft said: *for each policy site, check the predicate over the observations attributed to it.* That quantifies existentially over sites. Soundness requires universal quantification over *observations*.

The difference is not pedantic. A per-site predicate cannot distinguish **"no observation here"** from **"compliant here"**. So every mechanism that moves an observation off a named site — outlining, misattribution, a build without debug info — leaves a site with nothing attached, which the per-site rule reads as compliance. It *fails open*, silently.

Reproduced with extractor, provenance map and artifact all held fixed: one aarch64 build at `-Oz` yields VERIFIED under the per-site rule and a refusal under the corrected one.

The correction. Every observation of a policy-relevant kind in the artifact must be discharged by exactly one policy site. Undischarged observations are the failure condition; per-site predicates run only *after* that obligation is met.

The cost was measured, not estimated: across 42 fixtures × 2 targets × 2 optimization levels, 167 of 168 builds admissible — a refusal surface near 1%. It is affordable because it quantifies over policy-relevant observation *kinds*, not over instructions.

Remark 6.2 (The same mistake, one level down)**.** The original problem was that policy could not survive travelling downward. The first fix moved policy out of the way and rested the decision on the *site namespace* instead — but a site namespace is another thing the compiler is free to rewrite, so that repeats the original error against a different object. Provenance is demoted accordingly: it may localize a finding, never decide one.

6.6 What each stage owes

Stage	Obligation
Policy (top)	declare secrets, observer, releases, placement. Never travels down. It may be *composed* across translation units: that is composition at the top and crosses no level
Extraction (each level)	emit the observation set for that level. This is what earns the architecture: observations existing at no higher level, such as an outlined function's secret-indexed loads, are seen only here
Provenance	a **localization hint**, not a decision mechanism. Three-valued: present, absent, or present-but-unattested. The third state is load-bearing — confidently wrong provenance must degrade a diagnostic, never ship a leaky artifact marked verified
Decision (top)	**universal discharge**: every policy-relevant observation accounted for by exactly one site. Per-site predicates run afterwards
Differential	the actual soundness argument, not a supplement. Runs over *whole images*, never over policy-named sites, under one composed program-level policy
Refusal	absent provenance, unattested provenance, absent callee model, or any undischarged observation
Identity	binds *both* ends: input closure, link unit, target triple, subtarget features, codegen flags, compiler revision

Remark 6.3 (When this is worth the complexity, and when it is not)**.** For a *single pinned target tuple* a simpler architecture is equally sound — analyse the final artifact only, and re-supply policy at that level through a side file keyed on symbol and offset rather than carried in the IR. At one site, stack-slot merging, it is strictly *more* sound.

The staged architecture earns its complexity only when more than one target tuple ships. That is a checkable condition, and it should be checked before paying the cost rather than assumed.

Remark 6.4 (The role of markers changes, and shrinks)**.** Under Part 6.4 a declassification marker no longer has to survive to the binary. It only has to survive to the level where the *policy* is evaluated. That is a far weaker requirement than the one the attack findings demolished, and it is why the inversion is worth the engineering: it converts a problem with no known sound solution into one with a workable one.

6.7 The declassification marker, after the attacks

Markers still have a job: recording that a High-to-Low transition was *authorized*. The obvious realization — an opaque function the optimizer cannot see through — was attacked across five dimensions and does not survive. The failure is structural, not a tuning problem.

Why the function-shaped marker fails. A declaration must resolve at link time. The moment its definition is co-visible — LTO, ThinLTO, a header, the same translation unit — attribute inference deduces `returned` from the identity body, the optimizer replaces uses of the marker's result with its argument, and the release edge is severed *while the marker sits there decoratively.* A presence-based checker sees a marker and a clean path; the raw secret reaches the sink. Verified end-to-end through the real `clang` driver under full LTO.

That is the whole family: no definition means no link, and a definition means no protection.

What works instead: tied-register inline asm. There is no symbol, so there is nothing to define, nothing to link, and nothing for LTO to import.

```
%h = call T asm sideeffect "// sps.declassify id=$2",
                           "=r,0,i,~{memory}" (T %raw, i32 <ID>)
```

Each operand earns its place, and each was measured:

Operand	What it buys
`sideeffect`	prevents deletion when the result is unused, and prevents machine-level CSE merging two markers
`=r` output	puts the marker *on* the dataflow path. With no body, `returned` cannot be inferred — the exact failure of the function-shaped form
`0` tie	ties input to output, so the machine cost is zero. Measured 9 instructions marked versus 9 unmarked, no frame, no spills, tail call preserved
`i` on the id	the id must be an immediate. A non-constant id is a **hard instruction-selection error**, so identity recoverability is fail-closed by construction rather than by convention
`~{memory}`	no IR-level effect, but blocks post-register-allocation scheduling from reordering memory operations across the marker. Measured cost zero

Verified identical under `default<O1/O2/O3/Os/Oz>`, `lto<O2>`, `lto<O3>` and `thinlto<O2>`: marker present, id literal, sink consuming the marker's result — while the declassifier implementation is fully inlined and deleted.

The freeze point this forces. Post-LTO, post-optimization IR, immediately before instruction selection. The asm comment is dropped by the assembler — measured one occurrence in the `.s`, zero in the `.o`, zero in the linked binary — so anything later cannot see the marker at all. Where a shipped-binary attestation is genuinely required, the comment body can be replaced by a section note emitting a `(pc, release_id)` record with zero text instructions.

Two problems that do not go away. Stated as residual risk rather than buried:

- **`returned` is still expressible.** Inline asm does not structurally forbid it — the IR verifier accepts `returned` on an asm argument, and the bypass reproduces. It must be an explicit checker rule, because there is no IR-level prohibition.
- **A static marker count is not a release count.** Unrolling and inlining both multiply it. Occurrence cardinality has to be restated dynamically, or checked at an earlier freeze point — which is the counting gap of Remark 6.1 reappearing, unsolved.

Remark 6.5 (The general lesson)**.** The function-shaped marker failed because it asked the *linker* and the *optimizer* to respect a policy fact neither was told about. The inline-asm marker works because it asks for nothing: it is opaque by construction, not by permission. Every durable mechanism in this part has that shape — it survives because there is no decision point at which something could decide otherwise.

6.8 The differential check

Even with the inversion, run the analysis at each level independently and compare verdicts. Disagreement between levels is itself the finding:

Upper	Lower	Meaning
VERIFIED	UNSAFE	**laundering** — the lowering introduced the leak. No fixture in the corpus covers this direction
UNSAFE	VERIFIED	evidence destroyed; the leak was optimised out of view
VERIFIED	UNKNOWN	acceptable: monotone degradation
UNKNOWN	VERIFIED	suspicious — what fact appeared?

This is cheap. It needs no new theory, it reuses the existing corpus, and it is the mechanism that would have caught the inlined-declassifier case automatically rather than by inspection.

6.9 What this costs, and what it does not buy

The honest accounting.

It costs backend work. Extraction at machine-IR level is not an IR pass; it means reading or instrumenting the target layer, per target. The measurements already show four target tuples disagreeing on one input, so this is per-tuple work.

Provenance is best-effort. Debug-info `inlinedAt` chains do relate inlined instructions back to their original function — verified — but debug info is explicitly non-semantic and may be dropped. So the provenance map is a source of UNKNOWN, not a guarantee. That is acceptable precisely because UNKNOWN is a first-class outcome; it would be unacceptable in a three-valued design.

It is not a proof. This architecture makes the checker's answer be about the artifact that runs. It does not establish that the answer is right — that is still Theorem 5.4 and the assurance ladder of Part 5.

It does not close L4. Real latency, hardware isolation, and speculation remain outside any observation set you can extract statically.

6.10 One capture point, several extraction points

Two results in this document appear to disagree about where the artifact is frozen, and the disagreement is terminological. Resolving it fixes the most important architectural parameter, so it is worth doing explicitly.

The marker analysis concludes the freeze point is **post-LTO, immediately before instruction selection**, because the marker is not visible below that — measured, one occurrence in the `.s`, zero in the `.o`, zero in the linked binary. The inversion concludes observations must come from **the machine level**, because that is where `je` rather than `csel` is decided. Both are right. They are answers to different questions that were being given one name.

Split the concept:

Capture point	where the *annotated* artifact is snapshotted and hashed. Exactly one. Must be where policy is still readable: post-LTO, before instruction selection. Markers must survive to here and no further.
Extraction points	where *observations* are gathered. Several, at and below the capture point, including machine IR. Carry no policy at all.
Artifact identity	binds the capture point to every extraction point, so the report describes one compilation rather than a family.

The requirement on markers weakens accordingly, and this is the practical payoff: a marker must survive to the capture point, not to the binary. That is a far easier obligation, and it is why Part 6.7's inline-asm form is sufficient despite the assembler discarding it.

Remark 6.6 (What the identity must bind)**.** Binding the top-level module alone is not enough — the same module produced `je` on x86-64 and `csel` on aarch64 from identical IR. So the identity is a *tuple*: module hash, ordered pipeline digest,

target triple, target CPU and features, and optimization level. A verdict that does not name that tuple is not a claim about anything runnable. This is the same shape of requirement as per-coalition rows, and it lands on the same record.

6.11 Open problems

Irreducible: event multiplicity. This one is not an open problem to be solved later; it is a limit of the architecture, and naming it is the only honest handling.

Provenance can be *perfect* and the count still wrong. Measured: at `-O2 -g`, two `stp` store-pair instructions carrying entirely correct source attribution perform sixteen release events — vectorization at four to one, composed with store-pairing at two to one. Nothing is misattributed. The provenance map is exactly right. The count is off by a factor of eight.

Worse, the defect is structural rather than incidental. The decision quantifies over the *union* of observation sets, and set union is idempotent, so the formulation cannot count at all. A policy constraining release cardinality is therefore not checkable in this architecture as stated. It needs either a multiset, which changes the decision procedure, or a dynamic statement, which changes what is being claimed.

Until one of those is chosen, a release-count policy must be reported UNKNOWN, not approximated.

Two further problems remain open, stated rather than hidden.

Non-monotone cardinality. Because occurrence counts move both ways, a policy that constrains release count cannot be checked as an artifact invariant at a single level. It has to be a statement about the observation set, which means the extraction has to be trusted to be complete.

Multi-target verdicts. One artifact and four targets gave four different answers. Either a verdict names its target tuple, or the artifact must be verified for every tuple it will be built for — and the report needs a row per tuple, which is the same record-shape problem as coalitions.

7 From design to implementation

Every preceding part describes something that does not exist. This part is about the one piece that does, what it measured, and why the measurement matters more than the six parts before it.

7.1 What actually exists

It is worth being exact, because the distinction between a specification and a system is the easiest one to lose while writing a document like this.

Artifact	Size	Kind
MLIR fixtures	47	test data
C provenance files	42	test data
Actor examples	7	test data
Integration tests	8	test data
`check_harness.py`	929	metadata *validation*; checks fixtures are well-formed. Not analysis
`refusal_rate.py`	179	a scanner over IR text. Not analysis
`sps-scan.cpp`	174	**an actual analysis**: an MLIR `SparseDataFlowAnalysis` with a Low/High lattice

So: roughly a hundred test files, and of thirteen hundred lines of code exactly one hundred and seventy-four implement an information-flow analysis. Everything else validates, measures, or documents.

Remark 7.1 (Why the accounting matters)**.** It is easy to write "the architecture refuses 63% of functions" when what was measured is "63% of functions contain an operation with no source location." The first is a claim about a system; the second is a claim about clang's output plus a rule from a design document. Only the second was measured. Keeping the two apart is the difference between a result and a rationalisation.

7.2 The first real measurement

`sps-scan` was run over every fixture whose policy survives parsing — 17 of 47, the rest encoding their policy in SSA names that `mlir-opt` discards. It emits findings with a stable reason and a source location:

```
[secret-dependent-branch] llvm.cond_br @ loc(...:67:5)
[secret-to-public-sink]   llvm.store   @ loc(...:69:5)
findings: 2
```

The results sort into four groups, and all four are informative.

Caught, correctly. `predecessor_choice_blockarg.bad` yields two findings, including the MT-CM6 case where a secret selects a merge block argument with no secret on any SSA operand edge. `prefix_causal_release.bad` yields one. These are the countermodels a re-implementation would most plausibly fail, and a 174-line pass catches them.

Silent, correctly. Five fixtures yield zero findings: the public-allocation control, the offset-disjoint control, the overwritten-slot control, the repaired error oracle, and the constant-latency wolfSSL helper profile. A pass that reported everything as unsafe — the failure mode the corpus was previously unable to detect at all — fails here.

Imprecise, as predicted. Two precision controls fire, one finding each: the identical-successor branch and the value-cancelling `xor`.

This is **not** a defect. Those fixtures exist to document that a forward Low/High lattice *must* lose precision at exactly those sites, and each carries `l1 disposition: relational-required` saying so. The implementation is behaving the way Step 6 says it is required to. It is the first time a prediction in this document was confirmed by running something.

Missed, for known reasons. Three bad fixtures yield zero: the audience mismatch, the error oracle, and the affected wolfSSL helper profile. They need audience reasoning, release-policy conformance and a helper-latency contract respectively — none of which a taint pass implements. These are the shape of the remaining work, and they are visible only because something ran.

7.3 The argument for building before designing further

This document has been wrong, and corrected by measurement, six times:

Claim	Corrected by
whole-run release equality as the product's initial constraint	the spec forbids it; the encoding is unsound in the permissive direction
the declassification marker as an opaque function	attribute inference deducing `returned` once the definition is co-visible
per-site quantification in the decision rule	a build reporting VERIFIED under it and refusing under universal discharge
a register-only reduction of the laundering case	it did not reproduce; the conversion needs a memory operand
an arithmetic-mask repair	folded straight back into a select, byte-identical to the leaky twin
a refusal surface of about 1%	4.8% per observation, and 63% per function on real code

Six for six. Not one of these was caught by review, by internal consistency, or by re-reading the specification; every one was caught by running a tool and looking at the output. The corrected versions are better, but the pattern is the point: **in this problem domain, a design claim that has not been executed is not evidence.**

The remaining untested claims are all design claims. So the highest-value next action is not more design — it is to make the smallest real thing falsifiable.

7.4 The smallest useful next step

`sps-scan` is not referenced anywhere in the harness. Wiring it in converts the whole corpus from a specification into a test of an implementation:

1. Add it to the diagnostic `RUN` of the satisfiable bring-up gates. That turns "the analysis must emit this reason here" from an aspiration into a pass/fail.

2. Add it to the four precision controls under their declared RELATIONAL REQUIRED disposition. This is the first false-positive regression net the corpus has ever had — and the run above shows exactly which two controls it will legitimately trip.
3. Re-derive the refusal number from the analysis rather than from a text scanner, with a real policy scoping the observation set.

None of this needs the lowering architecture of Part 6, the `.ll`-versus-MLIR decision, or the thirty unmigrated fixtures. Those gate later layers. The diagnostic layer is buildable now, against a corpus that is already known to exercise it.

Remark 7.2 (What is durable)**.** The architecture in this document has been rewritten twice in a day and will likely change again. What has not changed, and will survive any rewrite, is the set of reproduced compiler behaviours, the fixtures that encode them, and the measurement scripts. Those are the assets. A design that cannot yet be executed should be weighted accordingly — including this one.

A Appendix: code map, report shape, and references

A.1 Where things live

All paths below are relative to `prototypes/compiler_harness/` in the repository root.

Path	Contents
`mlir/`	the 45 IR fixtures
`c/`	C reductions and provenance, one file per fixture family
`c/check_harness.py`	the metadata contract: the closed fixture inventory and every structural rule
`integration/`	metadata, witness-driver, import, and code-generation tests
`examples/actors/`	actor and ACL design examples; outside the enforced corpus
`mlir/L0_L1_L2_PIPELINE.md`	the fixture-by-fixture outcome table

Two orientation notes for a reader opening the corpus for the first time. The metadata contract is the authoritative description of what a fixture is allowed to claim — read it before reading fixtures. And the fixture inventory is *closed*: a fixture with no contract entry and a contract entry with no fixture both fail, so fixtures and registrations necessarily land together.

A.2 Running it

Command	Effect
`make check`	all IR and integration tests
`make check-mlir`	verifier and shape tests only
`make check-integration`	metadata, witness, import, code generation
`make bootstrap-lit`	create the pinned local test runner

Seventeen failures is the correct outcome, and every one of them should be a fixture that the metadata contract's `is_bad()` accepts — that is, one matching `.bad`, `_bad`, `lowered_bad`, or `target_bad`. All four markers are in use; `clangover_poly_frommsg.lowered_bad` matches only the third. A failure anywhere else is a real regression, and that distinction is worth checking mechanically rather than by eye.

A.3 The shape of a report

The walkthrough implies a record with more structure than a single verdict token. Collecting what each step required:

Field	Constraint that forces it
model status	PROVED carries no reason; a counterexample carries a witness; a refusal carries an ordered blocker set (Step 8)
deployment status	an independent axis: a model counterexample cannot be repaired by deployment evidence (Step 9)
policy review status	a third axis that by construction never changes either of the other two
diagnostic disposition	per site, from the five-valued domain of Step 6, plus a health flag; never a premise for safety
row key	one row per entry and coalition, over the derived closure (Step 3)
witness	replayable, and canonical enough that the discovery route is not observable in it (Step 8)

The three-status split is the part most likely to be collapsed under time pressure, and CONDITIONAL is precisely the projection of "model proved, deployment open." It is a derived value rather than a primitive alongside the other three statuses — which is why an obligation naming a target fact and one naming a backend fact should not share a field.

A.4 References

Verified against Crossref, dblp, and author- or proceedings-hosted copies. Publisher pages for USENIX, IEEE, ACM, and Springer were not directly retrievable, so those entries rest on the registration authority rather than the publisher's own page.

Self-composition and 2-safety. G. Barthe, P. R. D'Argenio, T. Rezk. *Secure Information Flow by Self-Composition.* CSFW 2004, 100–114; extended in *Math. Struct. in Comp. Sci.* 21(6):1207–1252, 2011. T. Terauchi, A. Aiken. *Secure Information Flow as a Safety Problem.* SAS 2005, 352–367. M. R. Clarkson, F. B. Schneider. *Hyperproperties.* CSF 2008, 51–65; *J. Computer Security* 18(6):1157–1210, 2010. N. Benton. *Simple Relational Correctness Proofs for Static Analyses and Program Transformations.* POPL 2004, 14–25.

Information-flow type systems. D. M. Volpano, C. E. Irvine, G. Smith. *A Sound Type System for Secure Flow Analysis. J. Computer Security* 4(2–3):167–187, 1996. A. Sabelfeld, A. C. Myers. *Language-Based Information-Flow Security. IEEE JSAC* 21(1):5–19, 2003.

Constant-time verification. J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, M. Emmi. *Verifying Constant-Time Implementations.* USENIX Security 2016, 53–70. *(ct-verif; toolchain unmaintained since 2017, benchmark corpus still current.)* Z. Zhang, G. Barthe. *CT-LLVM: Automatic Large-Scale Constant-Time Analysis.* IACR ePrint 2025/338. L.-A. Daniel, S. Bardin, T. Rezk. *Binsec/Rel.* IEEE S&P 2020, 1021–1038; extended in *ACM TOPS* 26(2), 2023. L. Cai, F. Song, T. Chen. *Towards Efficient Verification of Constant-Time Cryptographic Implementations.* FSE 2024.

Compiler preservation of hyperproperties. G. Barthe, S. Blazy, B. Grégoire, R. Hutin, V. Laporte, D. Pichardie, A. Trieu. *Formal Verification of a Constant-Time Preserving C Compiler. PACMPL* 4(POPL), Art. 7, 2020. J. Rosemann, S. Hack, D. Garg. *Non-interference Preserving Optimising Compilation. PACMPL* 9(OOPSLA2):1429–1454, 2025. C. Abate, R. Blanco, D. Garg, C. Hriţcu, M. Patrignani, J. Thibault. *Journey Beyond Full Abstraction.* CSF 2019, 256–271. M. Patrignani, A. Ahmed, D. Clarke. *Formal Approaches to Secure Compilation. ACM Computing Surveys* 51(6), 2019.

Mechanized LLVM and MLIR. J. Zhao, S. Nagarakatte, M. M. K. Martin, S. Zdancewic. *Formalizing the LLVM Intermediate Representation for Verified Program Transformations.* POPL 2012, 427–440. Y. Zakowski, C. Beck, I. Yoon, I. Zaichuk, V. Zaliva, S. Zdancewic. *Modular, Compositional, and Executable Formal Semantics for LLVM IR. PACMPL* 5(ICFP), 2021. *(current Vellvm.)* L. Silver, P. He, E. Cecchetti, A. K. Hirsch, S. Zdancewic. *Semantics for Noninterference with Interaction Trees.* ECOOP 2023, 29:1–29:29. N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, J. Regehr. *Alive2: Bounded Translation Validation for LLVM.* PLDI 2021, 65–79. M. Fehr, Y. Fan, H. Pompougnac, J. Regehr, T. Grosser. *First-Class Verification Dialects for MLIR. PACMPL* 9(PLDI):1466–1490, 2025. S. Bhat, A. Keizer, C. Hughes, A. Goens, T. Grosser. *Verifying Peephole Rewriting in SSA Compiler IRs.* ITP 2024, 9:1–9:20. *(lean-mlir.)*

Constant-time by construction. J. B. Almeida et al. *Jasmin: High-Assurance and High-Speed Cryptography.* CCS 2017, 1807–1823; *The Last Mile*, IEEE S&P 2020, 965–982. S. Cauligi et al. *FaCT: A DSL for Timing-Sensitive Computation.* PLDI 2019, 174–189.

A.5 Reading the arc backwards

One last framing, for a reader who wants the shortest possible summary of Part 3.

The checker never proves a program safe. It reduces "is this program safe" to "is this product unreachable," having first established that the question is well-posed: the policy is total, the artifact is in the fragment and is the one that shipped, and the admitted domain is non-empty. Every one of those preconditions is a place where a plausible shortcut yields a confident wrong answer, and the corpus exists because each shortcut is more tempting than it looks.